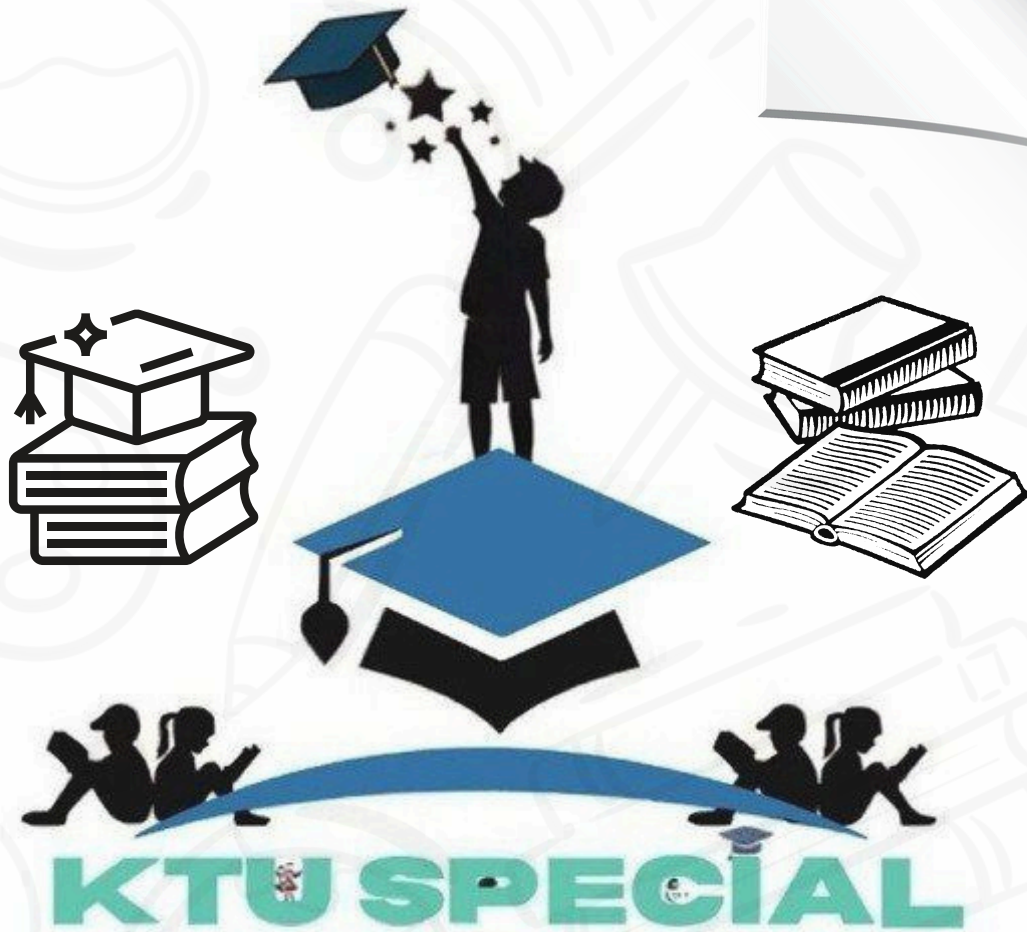




APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



We can together dream the B.tech



**APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY**
സാക്ഷാൽ അഗ്നി ജ്ഞാന സർവ്വകലാശാല

• **KTU STUDY MATERIALS**

• **SYLLABUS**

• **KTU LIVE NOTIFICATION**

• **SOLVED QUESTION PAPERS**

JOIN WITH US



WWW.KTUSPECIAL.IN



KTUSPECIAL



t.me/ktuspecial1

Syllabus

Fundamentals: Basic operational concepts, memory locations and addresses, instructions and instruction sequencing, instruction set principles modes, classifying instruction set architectures, memory addressing, type and size of operands, operations in the instruction set, instructions for control flow, encoding an instruction set, RISC architecture.

A computer is a digital system that executes instructions (programs) to process input and generate output.

Main Components:

- **CPU (Central Processing Unit):**
 - *Control Unit:* Coordinates all operations.
 - *ALU (Arithmetic Logic Unit):* Performs arithmetic and logic functions.
 - *Registers:* Small, fast storage units within CPU.
- **Memory:** Temporary (RAM) or permanent (ROM) storage for data and instructions.
- **Input Devices:** Devices like keyboard, mouse.
- **Output Devices:** Monitor, printer, etc.
- **System Buses:** They Connect CPU, memory, and I/O devices via:
 - *Data bus*
 - *Address bus*
 - *Control bus*

BASIC OPERATIONAL CONCEPTS:

1. Explain the basic operational concepts in transferring data between main memory and processor with diagram./ Explain in detail about the connections between the Processor and the Memory
2. with the help of a diagram.
3. Explain the basic structure of computers?
4. With a neat diagram ,explain the internal architecture of the CPU.(3marks,ktu,April 2018)
5. Write a short note on a. IR b. PC c. MAR d. MDR

Instruction Register (IR):

- It holds the instruction that is currently being executed.

Program Counter (PC):

Register containing the memory address of the next instruction to be fetched for execution- tracks execution of the program.

Memory Address Register (MAR):

It holds the address of the location to be accessed.

Memory Data Register (MDR):

It contains the data to be written into or read out of the address location.

- ❑ MAR and MDR facilitate the communication with memory.

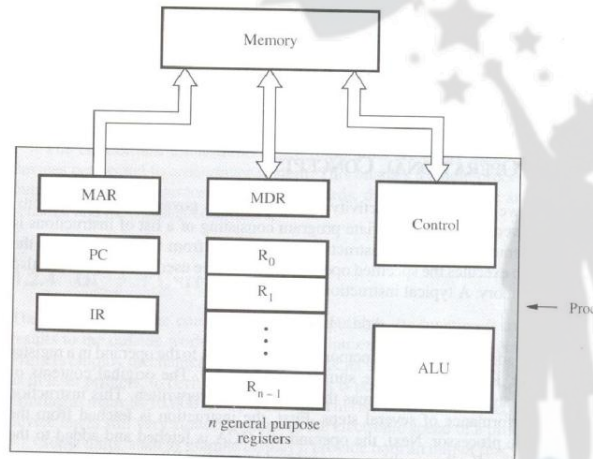


Figure 1.2 Connection between components

Operation Steps:

- ❑ The inputted program resides in memory. Execution starts when the Program Counter (PC) points to the address of the first instruction in the program, e.g., 2000
- ❑ [PC] is transferred to MAR and a Read control signal is sent to the memory.
- ❑ The Memory loads the addressed word (ie. First instruction eg, Add A, B) into MDR.
- [MDR] are transferred to IR. The instruction is ready to be decoded & executed.
- When operands are read into MDR, then sent to ALU for desired operation
- Then the result is sent to MDR and a Write Cycle is initiated. At some point of current execution, [PC] is incremented, and points to the next instruction to be executed

MEMORY LOCATIONS AND ADDRESSES

Memory Structure: Linear array of addressable storage cells.

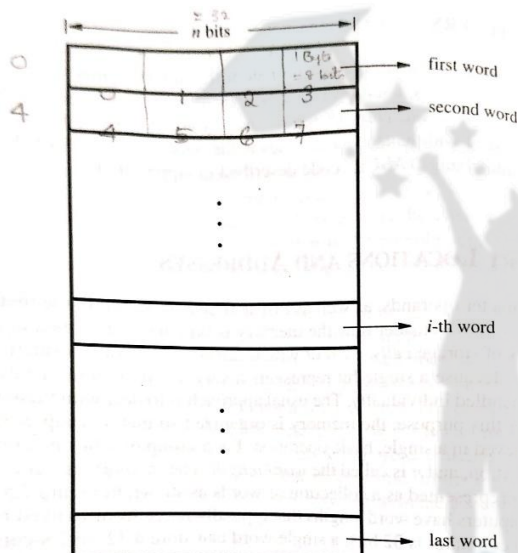
- Each location has a **unique address**.

Memory is organised so that a group of n BITS can be stored or retrieved in a single operation. Each group of n bits is referred to as a word of information, called word length.

Word length ranges from 16 to 64 bits. 8 bits is called a BYTE. Machine instructions may require one or more words for their representation.

n bits can be represented by 2^n memory locations (from 0 through $2^n - 1$) as the addresses of successive memory locations, constituting the Address Space (eg. 10 bit address generates an address space of 2¹⁰ Bytes, 1KB) MB, GB, TB

Memory stores first word, second words. Say each of $n=32$ bits.(fig)
 Most practical assignment is to have successive addresses refer to successive byte locations in memory->Byte Addressable



Differentiate between big-endian and little-endian byte assignments [3marks ,ktu]

- Two ways to assign byte address across words:
- Big-endian addressing assigns lower addresses to more significant (leftmost) bytes of word
- Little-endian addressing uses opposite order(used by Intel).
- Commercial computers use either approach, and some can support both approaches
- Byte locations have addresses 0, 1, 2, ...Addresses for 32-bit words are 0, 4, 8, ...

Word address	Byte address					Byte address			
0	0	1	2	3	0	3	2	1	0
4	4	5	6	7	4	7	6	5	4
$2^k - 4$	$2^k - 4$	$2^k - 3$	$2^k - 2$	$2^k - 1$	$2^k - 4$	$2^k - 1$	$2^k - 2$	$2^k - 3$	$2^k - 4$

(a) Big-endian assignment (b) Little-endian assignment

Problem: Consider a computer that has a byte addressable memory organized in 32-bit words according to the big-endian scheme. A program reads ASCII characters entered at a keyboard and stores them in successive byte locations, starting at location 100. Show the contents of the two memory words starting at location 100 after the word “SEMESTER” has been entered.

Step 1: Understand ASCII and Hex

ASCII stands for *American Standard Code for Information Interchange*.

- It's a character encoding standard used by computers to represent *text* and *control characters*.

Each character you type on a keyboard is stored as a byte (8 bits) using the **ASCII standard**. It assigns numerical values to characters 0-127, like A represented by 65 in decimal, a by 97,.. For example:

- 'S' is **ASCII decimal 83**, which is **0x53** in hex.
- This translation holds for all characters.

 Step 2: ASCII → Hex Mapping for "SEMESTER"

Character ASCII (Decimal) Hexadecimal

S	83	0x53
E	69	0x45
M	77	0x4D
E	69	0x45
S	83	0x53
T	84	0x54
E	69	0x45
R	82	0x52

So "SEMESTER" as a sequence of **hex bytes** looks like:

0x53 0x45 0x4D 0x45 0x53 0x54 0x45 0x52

Each byte takes 1 memory location. If stored starting at address 100, the placement in big-endian 32-bit words would be:

 Step 3: Group Bytes into 32-bit Words (4 bytes each)

- First Word (Addresses 100–103): 0x53 0x45 0x4D 0x45 → 0x53454D45
- Second Word (Addresses 104–107): 0x53 0x54 0x45 0x52 → 0x53544552

 Final Memory Representation

Address Range Word Content

100–103	0x53454D45
104–107	0x53544552

Number Systems Overview

1. *Decimal (Base 10)*

- Digits: 0–9
- Most familiar system—used in everyday counting.

2. *Binary (Base 2)*

- Digits: 0 and 1
- Used by computers to represent data and instructions.

3. *Hexadecimal (Base 16)*

- Digits: 0–9, then A–F (where A = 10, B = 11, ... F = 15)
- Often used in programming and memory addresses for compact representation of binary.

Conversion Guide

 Decimal ↔ Binary

- To convert decimal to binary: Divide the number by 2 repeatedly and record the remainders (bottom to top).

- Example: 13 → 1101
- To convert binary to decimal: Multiply each bit by 2^{position} and sum.
- Example: $1101 \rightarrow 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$

✓ Decimal ↔ Hexadecimal

- Decimal to Hex: Divide by 16 repeatedly and use remainders (just like binary but with base 16).
- Example: 26 → 1A
- Hex to Decimal: Multiply each digit by 16^{position} and sum.
- Example: $1A \rightarrow 1 \times 16^1 + 10 \times 16^0 = 16 + 10 = 26$

✓ Binary ↔ Hexadecimal

- Group binary digits into sets of 4 (from right), then convert each group to hex.
- Example: $11010111 \rightarrow 1101 \ 0111 \rightarrow D7$
- Hex to Binary: Convert each hex digit into 4-bit binary.
- Example: $D7 \rightarrow D = 1101, 7 = 0111 \rightarrow 11010111$

Binary ↔ Decimal

- *Binary to Decimal*: Multiply each bit by (2^{position}) , then sum.
- Example: $1011 \rightarrow 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$
- *Decimal to Binary*: Divide by 2 repeatedly, write remainders in reverse.
- Example: $11 \rightarrow 1011$

✓ *Binary ↔ Hexadecimal*

- *Binary to Hex*: Group bits into 4s from right, convert each group.
- Example: $110101 \rightarrow 0001 \ 1010 \ 1 \rightarrow 1A$
- *Hex to Binary*: Convert each hex digit to 4-bit binary.
- Example: $1A \rightarrow 0001 \ 1010 \rightarrow 11010$

✓ *Decimal ↔ Hexadecimal*

- *Decimal to Hex*: Divide by 16, write remainders using 0–9 & A–F.
- Example: $255 \rightarrow FF$
- *Hex to Decimal*: Multiply each digit by (16^{position}) , then sum.
- Example: $FF \rightarrow 15 \times 16 + 15 = 240 + 15 = 255$

Instruction Set Principles

Classification of Instruction Set Architectures (ISAs)....or Instruction Types

◇ 1. Stack-Based Architecture (Zero address instruction)

- **Operands** are implicitly on top of a stack.
- No need to mention operand addresses explicitly.
- **Instructions** use the stack for all operations (e.g., PUSH, POP, ADD).
- **Example Instruction:**
 - PUSH A
 - PUSH B
 - ADD ; Adds top two items on the stack
 - POP C ; Stores result in C
- **Pros:**

- Short instructions (no operand specifiers).
 - Simple hardware.
- **Cons:**
 - Harder to optimize.
 - Less flexible.

◇ 2. Accumulator-Based Architecture (one address)

- Uses a special register called the **accumulator** for one operand and stores the result there.
- **Example Instruction:**
- `LOAD A ; A → ACC`
- `ADD B ; ACC + B → ACC`
- `STORE C ; ACC → C`
- **Pros:**
 - Compact instructions.
 - Simpler control logic.
- **Cons:**
 - Frequent memory access.
 - Less parallelism.

◇ 3. Register-Memory Architecture (two address)

- One operand is in a register, the other in memory.
- Allows direct memory access in arithmetic instructions.
- **Example Instruction:**
- `ADD R1, A ; R1 + Memory[A] → R1`
- **Pros:**
 - Fewer instructions than register-register.
- **Cons:**
 - Memory access is slower.
 - May increase complexity of instructions.

◇ 4. Register-Register (Load/Store) Architecture

- All **arithmetic and logic operations are done only between registers.**
- Memory is accessed **only** through load/store instructions.
- **Example Instruction:**
- `LOAD R1, A`
- `LOAD R2, B`
- `ADD R3, R1, R2 (three address)`
- `STORE R3, C`
- **Examples:** MIPS, RISC-V, ARM (mostly), SPARC
- **Pros:**
 - Fast and simple pipeline.
 - Better optimization.
- **Cons:**

- More instructions required (due to extra load/store).

◇ 5. Memory-Memory Architecture

- All operands can reside in memory; no need for registers.
- **Example Instruction:**
- `ADD A, B, C ; Memory[B] + Memory[C] → Memory[A]`
- **Pros:**
 - Fewer instructions (no load/store).
- **Cons:**
 - More memory traffic → slower execution.
 - Complicated to pipeline.

Summary Table:

Architecture Type	Operand Sources	Example CPU	Comments
Stack	Stack	Java VM	Implicit operands
Accumulator	Accumulator + Memory	Early Intel (e.g., 8080)	Simple but outdated
Register-Memory	Register + Memory	x86	Mixed efficiency
Register-Register	Only Registers	MIPS, ARM, RISC-V	RISC style
Memory-Memory	Only Memory	VAX	Complex, rare today

Program Example

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

How is the operation X performed using:

- Three address instruction
- Two address instruction
- One address instruction
- Zero address

Expression: $X = (A+B)*(C+D)$

Three Address Instructions

R1, R2 are registers

```
ADD    R1,A, B,
ADD    R2, C, D,R
MUL    X,R1, R2
```

Two Address Instructions

```
ADD A,B
```

ADD C,D
MUL A,C
MOVE X,A

One Address Instructions

AC is accumulator

M[] is any memory location

M[T] is temporary location

LOAD A	AC = M[A]
ADD B	AC = AC + M[B]
STORE T	M[T] = AC
LOAD C	AC = M[C]
ADD D	AC = AC + M[D]
MUL T	AC = AC * M[T]
STORE X	M[X] = AC

Zero address

PUSH A ; Push A onto stack

PUSH B ; Push B onto stack

MUL ; Multiply top two values (A * B) and Stack pops A and B, pushes result A*B automatically

PUSH C ; Push C onto stack

PUSH D ; Push D onto stack

MUL ; Multiply top two values (C * D) and Stack pops C and D, pushes result C*D automatically

ADD ; Add the top two results: (A*B) + (C*D)

POP X ; Store result into X

Q. How is the operation $X = (A + B) * (C + D) / (E + F)$ is performed using:

- a) Three address instruction
- b) Two address instruction
- c) One address instruction

Instruction Set Modes: Addressing Modes (how operands are accessed)

Addressing modes define how the operand of an instruction is selected (from memory, register, immediate value, etc.). This is a core part of instruction set architecture.

Q. Illustrate with example, explain the different types of addressing modes in a RISC processor. (9 marks)

What is addressing mode? Explain the following addressing modes with example

(i) Immediate (ii) Indexed (iv) Auto increment (9) (KTU 2021)

With the help of examples, explain the different addressing modes.

(5 marks)

Addressing mode	Example instruction	Meaning	When used
Register	Add R4, R3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$	When a value is in a register.
Indexed indexed	Add R4, #3	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$	For constants.
	Add R4, 100(R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$	Accessing local variables (+ simulates register indirect, direct addressing modes).
indirect	Add R4, (R1)	$\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$	Accessing using a pointer or a computed address.
Indexed	Add R3, (R1, R2)	$\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount.
Direct or absolute	Add R1, (1001)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may
Autoincrement	Add R1, (R2)+	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$ $\text{Regs}[R2] \leftarrow \text{Regs}[R2] + d$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d.
Autodecrement	Add R1, -(R2)	$\text{Regs}[R2] \leftarrow \text{Regs}[R2] - d$ $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$	Same use as autoincrement. Autodecrement/-increment can also act as push/pop to implement a stack.

Here are the **main addressing modes**:

Addressing Mode	Description	Example
Immediate	Operand is part of the instruction	ADD R1, R2, #5 (add 5 to R2)
Register	Operand is in a register	MOV R1, R2
Direct (Absolute)	Address of operand is given directly	LOAD R1, 5000 (from mem[5000])
Indirect	Address of the operand is in a register or memory	LOAD R1, (R2) (R2 has address)
Indexed	Address of operand = base + index (often for arrays)	LOAD R1, 1000(R2)
Base + Index + Offset	Address of operand = base + index + offset	LOAD R1, 8(R2, R3)
Relative	Used for branching; PC-relative addressing Address of operand = offset + PC	JMP 40 (relative to PC)
Autoincrement	Depending on pre/post increment	SUB (R1)+, R2
Autodecrement	Depending on pre/post decrement	SUB -(R1), R2

Addressing Mode	Description	Example
Implied	Operand implied by instruction	CLR (clear accumulator)

ADDRESSING MODES

The different ways in which the location of an operand is specified in an instruction is called as **Addressing mode**.

Register

Absolute/ Direct

Immediate

Indirect

Indexed

Relative

Autoincrement

Autodecrement

1. Register Mode:

The operand is the contents of the processor register. **Eg: MOVE R1,R2**

2. Absolute Mode(Direct Mode):

The address location is given explicitly in the instruction. **Eg: MOVE LOC,R2**

Information from which memory address of the operand can be obtained is Effective Address
EA=LOC

3. Immediate mode :

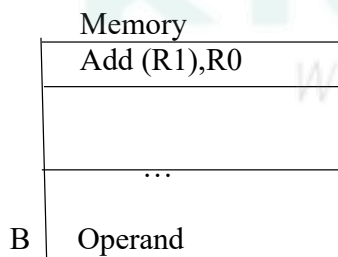
The operand is given explicitly in the instruction.

Eg: Move200_{immediate} ,R0 or Move #200,R0

4. Indirect Mode:

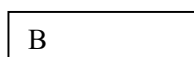
Instruction does not give the operand or its address explicitly. Instead it provides **information from which the new address of the operand can be determined.**

Eg. Add (R1), R0. R1 contains the address of an operand, EA=[Ri]



A

Register R1



5. Index Mode:

- The effective address of an operand is generated by adding a constant value to the contents of a register. Eg. $\text{AddX}(\text{Ri})$, R1 , X defines an offset, $\text{EA} = \text{X} + [\text{Ri}]$

Eg: $\text{Add } 20(\text{R1}), \text{R2}$ (eg. If R1 holds memory address 1000, $\text{EA} \Rightarrow 1000 + 20 = 1020$)

Index Mode	Assembler Syntax	Addressing Function
Index	$\text{X}(\text{Ri})$	$\text{EA} = [\text{Ri}] + \text{X}$
Base with Index	(Ri, Rj)	$\text{EA} = [\text{Ri}] + [\text{Rj}]$
Base with Index and offset	$\text{X}(\text{Ri}, \text{Rj})$	$\text{EA} = [\text{Ri}] + [\text{Rj}] + \text{X}$

6. Relative Addressing:

It is same as index mode. The difference is, instead of general purpose register, here we can use program counter(PC).

$$\text{EA} = [\text{PC}] + \text{X}$$

There are two additional modes: Auto-increment mode, Auto-decrement mode

Describe auto increment addressing mode with the help of an example.[3 marks, ktu]

7. Auto-increment mode:

After accessing the operand, the contents of this register is automatically incremented to point to the next item in the list.

Mode	Assembler syntax	Addressing Function
Auto-increment	$(\text{Ri})+$	$\text{EA} = [\text{Ri}]; \text{Increment Ri}$

8. Auto-decrement mode:

Before accessing the operand, the contents of this register is automatically decremented to point to the previous item in the list.

Mode	Assembler Syntax	Addressing Function
Auto-decrement	$-(\text{Ri})$	Decrement Ri ; $\text{EA} = [\text{Ri}]$

Questions

Registers R1 and R2 of a computer contains the decimal values 1000 and 3600. What is the effective address of the memory operand in each of the following instructions? (a) $\text{LW } 10(\text{R1}), \text{R2}$ (b) $\text{SW } \text{R3}, 10(\text{R1}), \text{R2}$ (c) $\text{ADD } -(\text{R1}), \text{R3}$ (d) $\text{SUB } (\text{R2})+, \text{R6}$ (e) $\text{MOV } \#5000, \text{R1}$ The word length of the processor is 4 bytes.

Let's determine the **Effective Address (EA)** for each instruction, considering:

- **R1 = 1000**
- **R2 = 3600**
- **Word length = 4 bytes** (this tells us each word is 4 bytes, but **EA is still in bytes**, unless stated otherwise)

We will evaluate each instruction assuming standard addressing modes.

☑ (a) `LW 10(R1), R2`

- **Addressing mode: Base + offset**
- $EA = R1 + 10 = 1000 + 10 = 1010$
- **Effective Address: 1010**

✓ **EA = 1010**

☑ (b) `SW R3, 10(R1, R2)`

- **Addressing mode: Base + Index + Offset**
- $EA = 10 + R1 + R2 = 10 + 1000 + 3600 = 4610$
- **Effective Address: 4610**

✓ **EA = 4610**

☑ (c) `ADD -(R1), R3`

- **Addressing mode: Autodecrement**
- Word size is 4 bytes → **R1 is decremented by 4**
- $R1 = 1000 - 4 = 996$
- **EA = 996**

✓ **EA = 996**

☑ (d) `SUB (R2)+, R6`

- **Addressing mode: Autoincrement**
- **EA = current value of R2 = 3600**
- Then, R2 is incremented by word size (4 bytes) → $R2 = 3604$

✓ **EA = 3600**

☑ (e) `MOV #5000, R1`

- **Addressing mode: Immediate**
- The literal value 5000 is directly moved into R1.
- No memory access → **No effective address**

✓ **EA = N/A (Immediate operand)**

List the advantages of using addressing modes? Justify with example the need for addressing modes in a RISC processor. (7)

Addressing modes are used to perform memory operations in a processor. Operands or instructions are fetched from the main memory using addressing modes. By decoding each addressing mode, processor is able to fetch the operand from the effective address locations. Fetching made simple, easy way to find or execute instruction, processor only needs to check the given address location, sequential search for instruction of operand is reduced, increase the time efficiency, increases the throughput, execution of operations are made simple using addressing modes

5. Register R1 and R2 of a computer contain the values 1800 and 3800. The word length of the processor is 4 bytes. Compute the effective address of memory operand in the following instruction

- a) **ADD 100(R2), R6.**
- b) **LOAD R6, 20(R1,R2)**
- c) **STORE -(R2), R6**
- d) **SUBTRACT (R2)+,R6**

Ans :

- b) Effective address = $100 + \text{Contents of R2} = 100 + 3800 = 3900.$
- c) Effective address = $20 + \text{Contents of R1} + \text{Contents of R2} = 20 + 1800 + 3800 = 5620.$
- d) Effective address = $\text{Contents of R2} - 4 = 3800 - 4 = 3796.$

Effective address = $\text{Contents of R2} + 4 = 3800 + 4 = 3804.....$

Q. The register R1 = 12, and R2= 13. The instruction ADD R1, R2 is in memory location 2000H.

After the execution of the instruction, what will be the value of PC, MAR, IR and R1?.

PC=2004H, MAR=2000H, IR= ADD R1, R2 and R1=25

Q. The register R1 = 200, and R2= 500. The instruction ADD R1, R2 is in memory location 5000H.

After the execution of the instruction, what will be the value of PC, MAR, MDR?

PC=5004H, MAR=5000H, MDR= **ADD R1, R2**

The register R1 = 18, and R2 = 20. The instruction ADD R1, R2 is in memory location 2020. After the execution of the instruction, what will be the values of PC, R1 and R2?

PC=2024

R1= 18

R2= 38

INSTRUCTIONS

What is Register Transfer Notation:

- Arbitrary names for locations in memory.
- []denote contents of a location
- Example: $R1 \leftarrow [A]$
(transfer from address A in memory to register R1)
- Example: $R1 \leftarrow [R2] + [R3]$
(add the contents of registers R2 and R3, place the sum in register R1)

Assembly Language Notation:

The **Assembly Language Notation** for the above examples are:

Move A , R1
Add R1, R2, R3

What are the Three, Two, One instruction types with examples.

Instruction Types	Syntax	Eg.
Three Address	Opcode Destination, Source1, Source2,	Add A,B,C Add values of memory locations B ,C & place the result into A.
Two Address	Opcode Destination, Source	Add A,B Add the values of A,B& place the result into A. A is overwritten.
One Address	Opcode Source	Add B Content of processor register Accumulator adds with content of B, and places sum in Acc $Acc \leftarrow Acc + B$ Other eg. Load B, Store B Load B means $Acc \leftarrow B$ Store B means reverse
Zero address	In Stack operations (Stack-Based Architecture)	POP Operand defined implicitly.

Explain instruction and instruction sequencing/ execution.

An instruction is a set of codes that the computer processor can understand. Has opcode and one or more operands

The execution of an instruction in a computer follows a series of well-defined **7 phases**.

◇ 1. Fetch Phase

- **Objective:** Retrieve the next instruction from memory.
- **Steps:**
 - The **Program Counter (PC)** holds the address of the next instruction.
 - This address is sent to the **Memory Address Register (MAR)**.
 - The instruction is fetched from memory and placed in the **Memory Data Register (MDR)** or **Instruction Register (IR)**.
 - The PC is updated to point to the next instruction (usually $PC + 1$ or $PC +$ instruction size).

◇ 2. Decode Phase

- **Objective:** Interpret the fetched instruction.
- **Steps:**
 - The instruction in the **Instruction Register (IR)** is decoded by the **Control Unit**.
 - The opcode (operation code) is identified.
 - The operands and the addressing modes are also determined.
 - Necessary signals are prepared for the next steps.

◇ 3. Evaluate Address Phase *(if needed)*

- **Objective:** Compute the effective address for memory access instructions.
- **Steps:**
 - Based on the addressing mode (e.g., direct, indirect, indexed), the effective address is calculated.
 - For example, if indexed addressing is used:
$$\text{Effective Address} = \text{Base Address} + \text{Index Register}$$

◇ 4. Operand Fetch Phase

- **Objective:** Fetch the required operands.
- **Steps:**
 - If the instruction needs data from memory or registers, it is fetched at this stage.
 - For register operands, data is read from register file.
 - For memory operands, the effective address is used to read data from memory.

◇ 5. Execute Phase

- **Objective:** Perform the operation.
- **Steps:**
 - The **ALU (Arithmetic Logic Unit)** performs the operation specified by the opcode (e.g., ADD, SUB).
 - This may involve arithmetic, logic, or shift operations.
 - For control instructions, this phase may involve setting the PC to a new address.

◇ 6. Memory Access Phase *(if needed)*

- **Objective:** Access memory for load/store instructions.
 - **Steps:**
 - If the instruction involves reading from or writing to memory (e.g., LOAD, STORE), the memory is accessed here using the effective address.
-

◇ 7. Write Back Phase

- **Objective:** Store the result of execution.
- **Steps:**
 - The result of the ALU operation or data fetched from memory is written back to a register or memory location as required.

Cycle Repeats:

After completing all the above phases for one instruction, the CPU begins the cycle again for the next instruction pointed to by the PC.

◇ Summary Table

Phase	Purpose
Fetch	Get instruction from memory
Decode	Identify operation and operands
Evaluate Address	Calculate address (if indirect/indexed)
Operand Fetch	Get data from registers or memory
Execute	Perform the operation
Memory Access	Read/write data in memory
Write Back	Store result
Interrupt Check	Detect and handle interrupts

Broadly, There are 2 main phases for Instruction Execution. They are,

1. Instruction Fetch:

The instruction is fetched from the memory location whose address is in PC. This is placed in IR.

2. Instruction Execution:

Instruction in IR is examined to determine whose operation is to be performed.

Program execution Steps:

- To begin executing a program, the address of first instruction must be placed in PC.
- The processor control circuits use the information in the PC to fetch & execute instructions one at a time in the order of increasing order.
- This is called **Straight line sequencing**. During the execution of each instruction, the PC is incremented by 4 to point the address of next instruction. (Assuming: **The word length of the processor is 4 bytes**)

	Address	m/g contents
Execution begins →	i	Mov R ₀ , A
	$i+4$	Add R ₀ , B
	$i+8$	Mov C, R ₀
		:
A		:
B		:
C		:

Fig: Program Execution for ADD C, A, B

$$C \leftarrow A+B$$

Assume that the memory has 32-bit word length and byte-addressable

- Place first instruction word at address i . Address i for first instruction placed in PC
- Remaining instructions are at $i+4$, $i+8$. During the execution of each instruction, the PC is incremented by 4 to point the address of next instruction.
- After execution of third instruction : Move C, R₀ ; PC contents are $i+12$

Branching:

Describe branching instruction?

- Branch instructions are those which change the normal sequence of execution.
- Sequence can be changed either conditionally or unconditionally.

Fig: Straight Line Sequencing Program

for adding
'n' numbers

i	Mov R ₀ , NUM1
$i+4$	Add R ₀ , NUM2
$i+8$	Add R ₀ , NUM3
:	:
.	.
	Add R ₀ , NUMn
	Mov SUM, R ₀
	:

Fig: Using Conditional Branching

i	Mov R ₁ , N
$i+4$	Clr R ₀
$i+8$ LOOP	Determine address of 'Next' & add to R ₀
:	:
.	.
	Decrement R ₁
	Branch > 0 LOOP
	Mov SUM, R ₀
	:

- We can illustrate the concept of branching with a program that adds a list of numbers

For **Straight Line Sequencing Program** that adds a list of numbers: Separate Add instructions are used to add each number NUM1...NUMn, to the contents of register R0.

After all the numbers have been added, the result is placed in memory location SUM.

For illustrating Conditional Branching, use a loop to add n numbers (Fig. on right)

- Assume that size of list, n, stored at location N
- Use register R1 as a counter, initialized to N
- Body of loop segment named LOOP (address $i+8$) includes the instruction
Decrement R1 to decrement counter in each loop pass
- Branch ≥ 0 goes to branch target LOOP if and as long as contents of R1 are greater than zero. (checks the immediately preceding instruction R1, ie. if $R1 \geq 0$). Therefore, this is a conditional branch. PC value is reset to address $i+8$, to perform the loop.
- A conditional branch instruction causes a branch only if a specified condition is satisfied. Branch instruction loads a new value into the PC, instead of the sequential address order.

In **UnConditional Branching**, branch does not need a specified condition to be satisfied. PC is loaded with the Branch Address (400 in the Eg. JMP 400)

CONDITIONAL CODES FLAGS:

What are condition codes flags? List the different condition codes flags.(3 marks)

(Exp:1.5 marks+ Examples 1.5 marks)

The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions. This is accomplished by recording the required information in individual bits, often called **condition code flags**.

These flags are usually grouped together in a special processor register called the **condition code register or status register**. Individual condition code flags are **set to 1 or cleared to 0**, depending on the outcome of the operation performed.

Four commonly used flags are :

- **N(negative)** Set to 1 if the result is negative; otherwise, cleared to 0
- **Z(zero)** Set to 1 if the result is 0; otherwise, cleared to 0
- **V(overflow)** Set to 1 if arithmetic overflow occurs; otherwise, cleared to 0
- **C(carry)** Set to 1 if a carry-out results from MSB bit; otherwise, cleared to 0
- **Eg. The instruction Branch > 0 is taken if N or Z flags are not 1.**

Give the CLASSIFICATION OF MICROPROCESSORS

These are categorized into RISC and CISC.

RISC: List characteristics of RISC?(3)

It stands for Reduced Instruction Set Computer. It is a type of microprocessor architecture that uses a small set of instructions of uniform length. These are simple instructions that are generally executed in one clock cycle. RISC chips are relatively simple to design and inexpensive. The setback of this design is that the computer has to repeatedly perform simple operations to execute a larger program having a large number of processing operations. **Fixed-Length Instructions in RISC**

- **Simplifies decoding:** Easy to identify instruction boundaries.
- **Speeds up fetching:** Fixed size (e.g., 32 bits) allows quick, consistent fetch cycles.
- **Enhances pipelining:** Uniform instruction size improves pipeline efficiency.
- **Reduces control complexity:** Enables fast, hardwired control logic.
- **Helps compiler optimization:** Easier instruction scheduling and generation.

Examples: MIPS, SPARC, POWER PC, etc.

CISC:

It stands for Complex Instruction Set Computer. These processors offer the users, hundreds of instructions of variable sizes. CISC architecture includes a complete set of special-purpose circuits that carry out these instructions at a very high speed. These instructions interact with memory by using complex addressing modes. CISC processors reduce the program size and hence lesser number of memory cycles are required to execute the programs. This increases the overall speed of execution.

Examples: Intel architecture, AMD, VAX

No. of Bits needed

Now, The size of the address space depends directly on the number of bits used to store an address.

Bit Count vs Addressable Locations

Here's how the number of bits affects the address space:

Bits Used	*Addressable Locations*
8 bits	$2^8 = 256$ Bytes
16 bits	$2^{16} = 65,536$ Bytes = 32KB
32 bits	$2^{32} \approx 4.3$ billion Bytes = 4GB
64 bits	$2^{64} \approx 18$ quintillion B

So, in a 32-bit system, you can theoretically access around 4 GB of memory,

How many bit address space will it be in a 4GB memory?

$4GB = 2^2 \times 2^{30} \dots (\text{power} = 2 + 30 = 32)$

Therefore.. 32-bit address space

Q. A computer system has a memory unit consisting of **1M words**, with each word being **32 bits** wide. The instruction format includes four fields:

1. An **operation code (opcode)**
2. A **mode field** to specify one of **nine addressing modes**
3. A **register address field** to select one of **128 processor registers**
4. A **memory address field**

Assuming each instruction occupies **one memory word**, determine the number of **bits required for each field** in the instruction format.

☒ Solution:

Total instruction size = **32 bits**

We need to divide these 32 bits among the four fields.

1. Memory Address Field

Memory size = **1M words** = 1×2^{20} Bytes

So, to address any word in memory, we need:

$$\log_2(2^{20}) = \boxed{20 \text{ bits}}$$

2. Register Address Field

Number of registers = $128 = 2^7$ Bytes, ie. 7 bits needed

3. Mode Field

Number of addressing modes = 9

$\log_2(9) \approx 3.17 \Rightarrow 4 \text{ bits}$ (next whole number)

4. Opcode Field

Now, Total bits = 32

Opcode bits + 20 + 7 + 4 = 32

Therefore, Remaining bits for opcode = $32 - 31 = 1 \text{ bit}$

♦ Type and Size of Operands

Usually the type of an operand—integer, single-precision floating point, character, and so on—effectively gives its size.

Common operand types include character (8 bits), half word (16 bits), word (32 bits), single-precision floating point (also 1 word), and double-precision floating point (2 words).

Integers are almost universally represented as two's complement binary numbers.

Characters are usually in ASCII, but the 16-bit Unicode (used in Java) is gaining popularity with the internationalization of computers.

Until the early 1980s, most computer manufacturers chose their own floating-point representation. Almost all computers since that time follow the same standard for floating point, the IEEE standard 754.

Some architectures provide operations on character strings, although such operations are usually quite limited and treat each byte in the string as a single character. Typical operations supported on character strings are comparisons and moves.

For business applications, some architectures support a decimal format, usually called *packed decimal* or *binary-coded decimal*—4 bits are used to encode the values 0 to 9, and 2 decimal digits are packed into each byte.

Numeric character strings are sometimes called *unpacked decimal*, and operations—called *packing* and *unpacking*—are usually provided for converting back and forth between them.

One reason to use decimal operands is to get results that exactly match decimal numbers, as some decimal fractions do not have an exact representation in binary. For example, 0.1010 is a simple fraction in decimal, but in binary it requires an infinite set of repeating digits:

0.0001100110011... 2.

Thus, calculations that are exact in decimal can be close but inexact in binary, which can be a problem for financial transactions.

Summary:

A. Operand Types: Common operand **data types**:

Type	Description	Examples
Integer	Signed or unsigned whole numbers	32-bit, 64-bit
Floating Point	Real numbers with decimals	IEEE 754 format
Character	ASCII or Unicode	8-bit, 16-bit
Boolean	True or false	1 bit (usually)
Addresses	Pointers to memory locations	32-bit, 64-bit

B. Operand Size: Different instruction sets support operands of **varying sizes**:

Size	Common Use
8-bit	Byte operations, characters
16-bit	Short integers
32-bit	Standard integers (int)
64-bit	Long integers, pointers
128-bit+	SIMD, floating-point vector

- **Instruction encoding** often includes bits to **specify operand type and size**.
- **RISC architectures** usually support **fixed sizes** (32 or 64-bit), whereas **CISC architectures** (like x86) support **multiple operand sizes**.

♦ Operations in the instruction set

This refers to the **types of actions** that the instruction set allows the CPU to perform. These are grouped into **categories** based on their function:

Operator type	Examples
Arithmetic and logical	Integer arithmetic and logical operations: add, subtract, and, or, multiply, divide
Data transfer	Loads-stores (move instructions on computers with memory addressing)
Control	Branch, jump, procedure call and return, traps
System	Operating system call, virtual memory management instructions
Floating point	Floating-point operations: add, multiply, divide, compare
Decimal	Decimal add, decimal multiply, decimal-to-character conversions
String	String move, string compare, string search
Graphics	Pixel and vertex operations, compression/decompression operations

A. Data Transfer Instructions

- Move data between memory, registers, or I/O.
- Examples: MOV, LOAD, STORE, PUSH, POP
- Example:
 - `MOV R1, R2` ; Copy content of R2 to R1
 - `LOAD R1, 5000` ; Load from memory address 5000 into R1

B. Arithmetic Instructions

- Perform mathematical operations.
- Examples: ADD, SUB, MUL, DIV, INC, DEC
- Example:
 - `ADD R1, R2` ; $R1 = R1 + R2$

C. Logical Instructions

- Perform bitwise operations.
- Examples: AND, OR, XOR, NOT, SHIFT, ROTATE
- Example:
 - `AND R1, R2` ; $R1 = R1 \& R2$

D. Comparison Instructions

- Compare two values and set flags (used by control flow).
- Examples: CMP, TEST
- Example:
 - `CMP R1, R2` ; Compare R1 and R2

E. Input/Output Instructions

- Transfer data between CPU and I/O devices.
- Examples: IN, OUT

- Example:
- `IN R1, PORT1`

F. Control Transfer Instructions *(Covered more in next section)*

Instructions for Control Flow

Control flow instructions determine **how the execution of a program proceeds**, especially when the flow deviates from the normal sequential path.

◇ 1. Types of Control Flow Instructions

☑ Conditional Branches

- Branching occurs **only if a condition is true**.
- Common in **loops, if-statements**, and **comparisons**.
- Efficient execution depends on how conditions are tested (often with zero comparisons).

☑ Jumps

- **Unconditional transfers** to a different memory location.
- Often used for **loop exits, goto statements**, or **infinite loops**.

☑ Procedure Calls

- Used to call a **subroutine or function**.
- Involves **saving the return address**, often in a special link register or GPR (general-purpose register).
- May also require saving caller/callee registers based on calling convention.

☑ Procedure Returns

- Returns control to the instruction after the call.
 - Requires **retrieving the return address**, often using **register indirect jumps**.
-

◇ 2. Addressing Modes for Control Flow

☑ PC-Relative Addressing

- Target address = **PC + displacement**.
- Efficient because:
 - **Saves instruction bits**.
 - Supports **position-independent code** (used in shared libraries, dynamic linking).
 - Works well when the target is **near** the branch instruction.

☑ Register Indirect Addressing

- Target address is in a **register**.
 - Used when the destination is not known at compile time.
 - Enables **dynamic dispatch**, useful for:
 - `switch/case` statements
 - **Virtual functions** in OOP (e.g., Java, C++) (which allow different routines to be called depending on the type of the argument)
 - **Function pointers** in C/C++
 - **Dynamic libraries** loaded at runtime
-

◇ 3. Conditional Branch Evaluation Techniques

- Branch conditions can be handled by:
 - **Separate compare and branch**

- **Compare-and-branch instructions**
- **Condition code flags**
- **Figure A. The major methods for evaluating branch conditions, their advantages, and their disadvantages**

Name	Examples	How condition is tested	Advantages	Disadvantages
Condition code (CC)	80x86, ARM, PowerPC, SPARC, SuperH	Tests special bits set by ALU operations, possibly under program control.	Sometimes condition is set for free.	CC is extra state. Condition codes constrain the ordering of instructions since they pass information from one instruction to a branch.
Condition register	Alpha, MIPS	Tests arbitrary register with the result of a comparison.	Simple.	Uses up a register.
Compare and branch	PA-RISC, VAX	Compare is part of the branch. Often compare is limited to subset.	One instruction rather than two for a branch.	May be too much work per instruction for pipelined execution.

◇ 4. Procedure Call Conventions

☑ *Caller Save vs. Callee Save*

- **Caller Save:**
 - Caller saves any registers it wants preserved.
 - Called function is free to use those registers.
- **Callee Save:**
 - Callee saves registers it wants to use.
 - Caller doesn't worry about saving them.

➔ Most modern systems **use both**, defined in the **ABI (Application Binary Interface)**.

➔ Used to handle **global variables**, **separate compilation**, and **register allocation** reliably.

Encoding an Instruction Set

Instruction encoding defines **how instructions are represented in binary** and affects both **performance** and **code size**.

◇ 1. Key Components of Instruction Encoding

- **Opcod**: Specifies the operation (e.g., ADD, LOAD).
- **Operand fields**: Identify source/destination registers.
- **Addressing mode fields**: Define how memory addresses are interpreted.

◇ 2. Design Considerations in Encoding

1. **More registers and addressing modes** → more bits needed.
2. **Longer instructions** → larger code size.
3. **Pipelining** prefers instructions with predictable formats.
4. **Ease of decoding** → faster CPU cycles.

◇ 3. Types of Instruction Encoding

add EAX,1000(EBX)

Operation and no. of operands	Address specifier 1	Address field 1	...	Address specifier n	Address field n
----------------------------------	------------------------	--------------------	-----	--------------------------	----------------------

(a) Variable (e.g., Intel 80x86, VAX)

Operation	Address field 1	Address field 2	Address field 3
-----------	--------------------	--------------------	--------------------

(b) Fixed (e.g., Alpha, ARM, MIPS, PowerPC, SPARC, SuperH)

Operation	Address specifier	Address field
-----------	----------------------	------------------

Operation	Address specifier 1	Address specifier 2	Address field
-----------	------------------------	------------------------	------------------

Operation	Address specifier	Address field 1	Address field 2
-----------	----------------------	--------------------	--------------------

(c) Hybrid (e.g., IBM 360/370, MIPS16, Thumb, TI TMS320C54x)

Figure: Three basic variations in instruction encoding: variable length, fixed length, and hybrid

☑ 1. Variable-Length Encoding

- Instruction sizes vary (e.g., 1 to 17 bytes on x86).
- **Advantages:**
 - Smaller code size.
 - Efficient use of memory.
- **Disadvantages:**
 - Complex hardware for decoding.
 - Slower pipelining.

☑ 2. Fixed-Length Encoding

- All instructions are the same length (e.g., 32 bits in RISC).
- **Advantages:**
 - Easy and fast decoding.
 - Good for pipelining and parallel execution.
- **Disadvantages:**
 - Larger code size due to unused fields.

☑ 3. Hybrid Encoding

- Mixes fixed and variable sizes (e.g., ARM Thumb, MIPS16).
- Common in **embedded systems** where code size matters.
- Balances compact code and decoding ease.

◇ 4. Real-World Examples

☑ ARM Thumb / MIPS16

- Use **16-bit and 32-bit** instructions.
- Trade-offs:

- 16-bit: smaller, less powerful instructions.
 - 32-bit: richer, but larger instructions.
- Achieves **~40% code size reduction**.

✓ **IBM CodePack**

- Compresses instructions before loading.
- Decompresses only when fetched from memory.
- Keeps the instruction cache in full 32-bit form.
- Reduces code size without changing compiler behavior.

✓ **Hitachi SuperH**

- Fixed 16-bit instruction set.
- 16 registers instead of 32.
- Ideal for **embedded applications**.

← **Summary of Instruction Encoding:**

Encoding Type	Code Size	Decoding Speed	Use Case
Variable-Length	Small	Slower	CISC (x86), legacy systems
Fixed-Length	Larger	Faster	RISC (MIPS, SPARC, ARM)
Hybrid	Balanced	Moderate	Embedded systems (ARM Thumb, MIPS16)

Explain RISC Architecture.

The MIPS architecture is a classic example of a **RISC (Reduced Instruction Set Computer)** design. It is known for its simplicity, regularity, and efficiency — especially in pipelined processors and compiler optimization.

◆ 1. Core Characteristics of MIPS

- **Load-store architecture:**
All data transfers between memory and CPU registers are performed using **load** and **store** instructions. Arithmetic and logic operations only work on registers.
 - **Fixed-length 32-bit instructions:**
Every instruction is 4 bytes long, making it easy to **decode** and **pipeline**.
 - **Simple instruction set:**
Only essential instructions are provided, reducing hardware complexity.
 - **Designed for pipelining:**
Instruction format and operations are optimized for efficient execution in pipelined CPUs.
 - **Compiler-friendly design:**
Its regular structure allows compilers to generate efficient code easily.
-

◆ 2. Registers in MIPS

✓ General-Purpose Registers (GPRs)

- 32 registers: R0 to R31, each 64 bits wide in MIPS64
- R0 is **always zero** — writing to it has no effect
- Used for integer operations and addresses

✓ Floating-Point Registers (FPRs)

- 32 registers: F0 to F31
- Support **single (32-bit)** and **double (64-bit)** precision
- Each 64-bit register can hold two 32-bit floats for **paired single (PS)** operations

◆ 3. Data Types

- **Integer types:**
 - Byte (8-bit), Halfword (16-bit), Word (32-bit), Doubleword (64-bit)
- **Floating-point types:**
 - Single precision (32-bit), Double precision (64-bit)

◆ 4. Addressing Modes in MIPS

- **Displacement (base + offset)** – Common in load/store:
`LW R1, 4(R2)` means $R1 \leftarrow \text{Mem}[R2 + 4]$
- **Immediate** – Constants embedded in the instruction
- **Register indirect via R0** or zero offset used to simulate other modes

◆ 5. Instruction Format

MIPS instructions are categorized into three main types, each designed for different kinds of operations:

🧠 R-type (Register type):

- Used for: Operations between registers

1234 I-type (Immediate type):

- Used for: Operations involving constants or memory access

- Fields include: opcode, rs, rt, immediate (16-bit value)

- Example instructions:

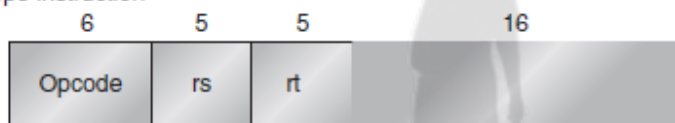
- `addi $t0, $t1, 10` → Add 10 to \$t1 and store in \$t0 , (i stands for immediate, \$ indicate Registers

📦 J-type (Jump type):

- Used for: Jumping to specific instruction addresses

- **All instructions are 32 bit:** 6-bit opcode + fixed-length fields for register names, immediates, etc.
- Easy to decode for pipelining and hardware simplification
- Example:
- R-type instruction is 32 bits and broken down into six fields:
- [opcode][rs][rt][rd][shamt][funct]
- rs - First source register
- rt - Second source register
- rd - Destination register
- shamt - Shift amount (used in shift instructions only)
- funct - Function code (specifies the actual operation)

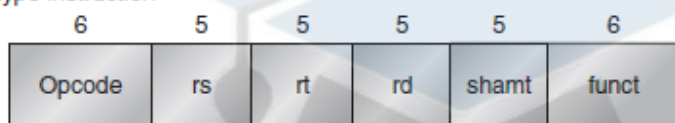
I-type instruction



Encodes: Loads and stores of bytes, half words, words, double words. All immediates ($rt \leftarrow rs \text{ op immediate}$)

Conditional branch instructions (rs is register, rd unused)
Jump register, jump and link register
($rd=0$, $rs=\text{destination}$, $\text{immediate}=0$)

R-type instruction



Register-register ALU operations: $rd \leftarrow rs \text{ funct } rt$
Function encodes the data path operation: Add, Sub, ...
Read/write special registers and moves

J-type instruction



Jump and jump and link
Trap and return from exception

Figure A.22 Instruction layout for MIPS. All instructions are encoded in one of three types, with common fields in the same location in each format.

◆ 6. Instruction Categories

(Write 2-3 examples from the tables given below)

✓ Load/Store Instructions

- LW, SW, LB, SB, etc.

- Only these access memory

Example instruction	Instruction name	Meaning
LD R1, 30(R2)	Load double word	$\text{Regs}[R1] \leftarrow_{64} \text{Mem}[30 + \text{Regs}[R2]]$
LD R1, 1000(R0)	Load double word	$\text{Regs}[R1] \leftarrow_{64} \text{Mem}[1000 + 0]$
LW R1, 60(R2)	Load word	$\text{Regs}[R1] \leftarrow_{64} (\text{Mem}[60 + \text{Regs}[R2]])_0^{32} \text{ ## Mem}[60 + \text{Regs}[R2]]$
LB R1, 40(R3)	Load byte	$\text{Regs}[R1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[R3]])_0^{56} \text{ ## Mem}[40 + \text{Regs}[R3]]$
LBU R1, 40(R3)	Load byte unsigned	$\text{Regs}[R1] \leftarrow_{64} 0^{56} \text{ ## Mem}[40 + \text{Regs}[R3]]$
LH R1, 40(R3)	Load half word	$\text{Regs}[R1] \leftarrow_{64} (\text{Mem}[40 + \text{Regs}[R3]])_0^{48} \text{ ## Mem}[40 + \text{Regs}[R3]] \text{ ## Mem}[41 + \text{Regs}[R3]]$
L.S F0, 50(R3)	Load FP single	$\text{Regs}[F0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[R3]] \text{ ## } 0^{32}$
L.D F0, 50(R2)	Load FP double	$\text{Regs}[F0] \leftarrow_{64} \text{Mem}[50 + \text{Regs}[R2]]$
SD R3, 500(R4)	Store double word	$\text{Mem}[500 + \text{Regs}[R4]] \leftarrow_{64} \text{Regs}[R3]$
SW R3, 500(R4)	Store word	$\text{Mem}[500 + \text{Regs}[R4]] \leftarrow_{32} \text{Regs}[R3]_{32..63}$
S.S F0, 40(R3)	Store FP single	$\text{Mem}[40 + \text{Regs}[R3]] \leftarrow_{32} \text{Regs}[F0]_{0..31}$
S.D F0, 40(R3)	Store FP double	$\text{Mem}[40 + \text{Regs}[R3]] \leftarrow_{64} \text{Regs}[F0]$
SH R3, 502(R2)	Store half	$\text{Mem}[502 + \text{Regs}[R2]] \leftarrow_{16} \text{Regs}[R3]_{48..63}$
SB R2, 41(R3)	Store byte	$\text{Mem}[41 + \text{Regs}[R3]] \leftarrow_8 \text{Regs}[R2]_{56..63}$

Figure A.23 The load and store instructions in MIPS. All use a single addressing mode and require that the memory value be aligned. Of course, both loads and stores are available for all the data types shown.

✓ ALU Instructions

Example instruction	Instruction name	Meaning
DADDU R1, R2, R3	Add unsigned	$\text{Regs}[R1] \leftarrow \text{Regs}[R2] + \text{Regs}[R3]$
DADDIU R1, R2, #3	Add immediate unsigned	$\text{Regs}[R1] \leftarrow \text{Regs}[R2] + 3$
LUI R1, #42	Load upper immediate	$\text{Regs}[R1] \leftarrow 0^{32} \text{ ## } 42 \text{ ## } 0^{16}$
DSLL R1, R2, #5	Shift left logical	$\text{Regs}[R1] \leftarrow \text{Regs}[R2] \ll 5$
SLT R1, R2, R3	Set less than	$\text{if } (\text{Regs}[R2] < \text{Regs}[R3])$ $\text{Regs}[R1] \leftarrow 1 \text{ else } \text{Regs}[R1] \leftarrow 0$

Figure A.24 Examples of arithmetic/logical instructions on MIPS, both with and without immediates.

- Arithmetic: ADD, SUB, MUL, DIV
- Logical: AND, OR, XOR, NOR
- Shift: SLL, SRL, SRA
- Immediate versions: ADDI, ORI, ANDI, etc.

✓ Control Flow Instructions

- **Conditional Branches:**
 - BEQ, BNE, BGTZ, etc.
 - Use 16-bit PC-relative offsets
- **Unconditional Jumps:**

- J, JAL (Jump and Link)
- JR, JALR (register-based)
- Return address is stored in R31

Example instruction	Instruction name	Meaning
J name	Jump	$PC_{36..63} \leftarrow \text{name}$
JAL name	Jump and link	$\text{Regs}[R31] \leftarrow PC+8$; $PC_{36..63} \leftarrow \text{name}$; $((PC+4)-2^{27}) \leq \text{name} < ((PC+4)+2^{27})$
JALR R2	Jump and link register	$\text{Regs}[R31] \leftarrow PC+8$; $PC \leftarrow \text{Regs}[R2]$
JR R3	Jump register	$PC \leftarrow \text{Regs}[R3]$
BEQZ R4, name	Branch equal zero	if $(\text{Regs}[R4] == 0)$ $PC \leftarrow \text{name}$; $((PC+4)-2^{17}) \leq \text{name} < ((PC+4)+2^{17})$
BNE R3, R4, name	Branch not equal zero	if $(\text{Regs}[R3] \neq \text{Regs}[R4])$ $PC \leftarrow \text{name}$; $((PC+4)-2^{17}) \leq \text{name} < ((PC+4)+2^{17})$
MOVZ R1, R2, R3	Conditional move if zero	if $(\text{Regs}[R3] == 0)$ $\text{Regs}[R1] \leftarrow \text{Regs}[R2]$

Figure A.25 Typical control flow instructions in MIPS. All control instructions, except jumps to an address in a register, are PC-relative. Note that the branch distances are longer than the address field would suggest; since MIPS instructions are all 32 bits long, the byte branch address is multiplied by 4 to get a longer distance.

✓ Floating-Point Instructions

- Arithmetic: ADD.S, ADD.D, MUL.S, MUL.D, etc.
- Data movement: MOV.S, MOV.D, MFC1, DMFC1, etc.
- Compare and conditional branches: BC1T, BC1F

◆ 7. Special Operations

- **LUI (Load Upper Immediate):**
Loads a constant into the upper half of a register.
- **Synthesized instructions:**
 - LI (load immediate) = ADDI R, R0, imm
 - MOV = ADD R1, R2, R0
- **Multiply-Add (MADD) and paired-single operations** are available for graphics/multimedia performance.

◆ 8. Procedure Calls and Returns

- **JAL (Jump and Link):**
Stores the return address in R31

- **JR R31:**
Returns from a procedure
- No stack built-in — the compiler manages **stack frames** and **register saving**

◆ 9. Endianess and Memory

- Byte-addressable memory with **64-bit addressing**
- Supports **Big Endian** or **Little Endian** via a mode bit
- Aligned accesses are **mandatory**

◆ 10. Extensions & Variants

- **MIPS16:**
16-bit compact instructions for embedded use — smaller code size
- **Paired-single** operations:
SIMD-like performance using 64-bit registers to process two 32-bit floats at once

✓ Summary of MIPS Features

Feature	Description
ISA Type	RISC (Reduced Instruction Set Computer)
Register File	32 GPRs (64-bit), 32 FPRs (64-bit)
Instruction Format	Fixed 32-bit width
Addressing Modes	Immediate, Displacement
Memory Access	Load/Store only; byte-addressable
Control Flow	PC-relative branches, register-based jumps
Pipelining	Optimized via uniform instruction format
Procedure Calls	JAL / JR; return address in R31
Floating-Point Support	IEEE 754, single & double precision, paired-single
Use Cases	General-purpose CPUs, embedded systems, teaching

Justification:

While increasing the number of pipeline stages can improve performance, it does not always decrease the execution time of a program. Here's why:

☒ Advantages of More Pipeline Stages:

Higher clock speed: Each stage does less work, so the clock cycle time can be reduced.

Increased instruction throughput: More instructions can be processed simultaneously in different stages.

☒ But it also introduces drawbacks:

Increased pipeline overhead:

Each stage needs an inter-stage buffer (register), adding latency.

More stages = more overhead due to these pipeline registers.

Higher chance of hazards:

Data hazards, control hazards, and structural hazards become more difficult to handle.

Branch instructions, for example, can cause more pipeline stalls in a deeper pipeline.

Diminishing returns:

After a certain point, increasing the number of stages doesn't significantly improve performance.

Eventually, the overhead and complexity outweigh the benefits.

Increased penalty on pipeline stalls and flushes:

In deeper pipelines, when a stall or branch misprediction occurs, more stages have to be flushed, wasting more cycles.

Types of pipeline

- Uniform delay/ Linear pipeline

In this type of pipeline, all the stages will take same time (T_p) to complete an operation.

- Non-Uniform/Non-linear delay pipeline

In this type of pipeline, different stages take different time to complete an operation.

In this type of pipeline, **Pipeline Time (T_p) = Maximum(Stage Delay)**

For example, if there are 4 stages with delays, 1 ns, 2 ns, 3 ns, and 4 ns, then

$$T_p = \text{Maximum}(1 \text{ ns}, 2 \text{ ns}, 3 \text{ ns}, 4 \text{ ns}) = 4 \text{ ns}$$

If buffers are included between the stages,

$$T_p = \text{Maximum}(\text{Stage delay}) + \text{Buffer delay}$$

Q : Consider a 4 segment pipeline with stage delays (2 ns, 8 ns, 3 ns, 10 ns). Find the time taken to execute 100 tasks in the above pipeline.

Solution : As the above pipeline is a non-linear pipeline,

$$T_p = \max(2, 8, 3, 10) = 10 \text{ ns}$$

$$\text{We know that } ET_{\text{pipeline}} = (k + n - 1) T_p = (4 + 100 - 1) 10 \text{ ns} = 1030 \text{ ns}$$

CLASSIFICATION OF PIPELINE PROCESSORS

KTU SPECIAL CONNECT WITH US



WHATSAPP GROUP



FOLLOW US NOW



TELEGRAM CHANNEL



SUBSCRIBE NOW



www.ktuspecial.in

SUBSCRIBE



FOLLOW US



Visit Website

